

04. バックエンドAPI設計書

Version 1.3 / 2026-08-18 / FastAPI (Python)

1. エンドポイント一覧

1.1 Public(認証なし・公開)

Method / Endpoint	認証	処理概要
GET /api/v1/public/availability	なし(レート制限)	卓ごとの空席状況(label・capacity・occupiedのみ)を返却

1.2 Customer(顧客)

Method / Endpoint	認証	処理概要
GET /api/v1/tables/{table_id}/session	なし(レート制限)	Active卓の session_token と order_id を返却
GET /api/v1/products	なし	品切れを除いたメニュー一覧を取得
GET /api/v1/orders/{order_id}/history	X-Session-Token	注文履歴と参考小計金額を取得
POST /api/v1/orders/{order_id}/items	X-Session-Token	カートの注文情報をDBへ書き込む
POST /api/v1/orders/{order_id}/call	X-Session-Token	is_calling と call_reason を更新し、call_events へ記録

1.3 Staff(店舗)

Method / Endpoint	認証	処理概要
POST /api/v1/staff/login	なし(レート制限)	スタッフ認証。JWT発行
POST /api/v1/staff/logout	Bearer	ログアウト(クライアント側トークン破棄)
GET /api/v1/tables	Bearer	卓マスター一覧・座標・現在の状態(呼出理由を含む)を取得
GET /api/v1/staff/pending_items	Bearer	全卓の未提供明細を取得
GET /api/v1/staff/products	Bearer	品切れ・価格管理用の全商品一覧を取得
PATCH /api/v1/staff/products/{id}/sold_out	Bearer	品切れ状態(is_sold_out)の切替
PATCH /api/v1/staff/products/{id}/price	Bearer	メニュー価格(税込)の更新
PATCH /api/v1/tables/{table_id}/activate	Bearer	開栓。新セッション発行。二重開栓時は409
PATCH /api/v1/order_items/{item_id}/status	Bearer	提供完了 / Undo / 取消
POST /api/v1/orders/{order_id}/custom_items	Bearer	is_custom=true として明細追加
PATCH /api/v1/orders/{order_id}/call/resolve	Bearer	呼び出しの解除(卓はActiveのまま)
PATCH /api/v1/tables/{table_id}/clear	Bearer	会計済。status='Deactivated'

1.4 Batch

Method / Endpoint	認証	処理概要
POST /api/v1/jobs/daily_clear	X-Cron-Secret	全Active卓の強制クリア処理

2. 主要シーケンス

2.1 注文～提供

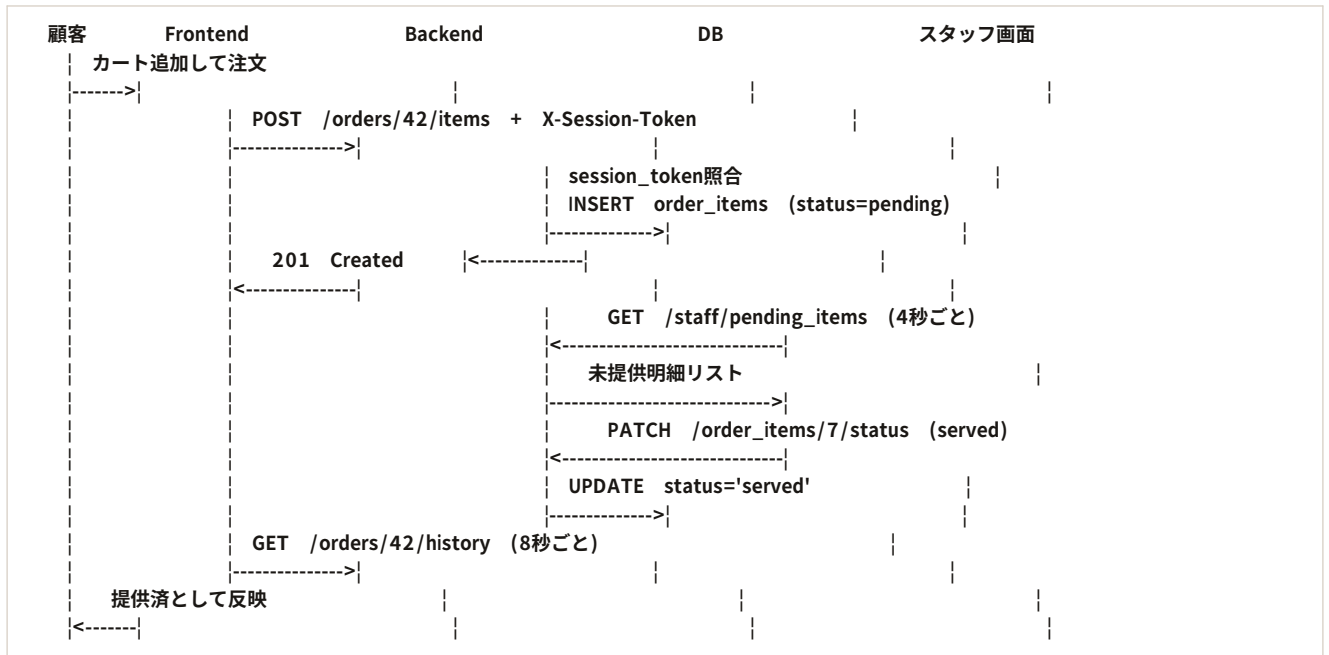


図1 注文～提供シーケンス

2.2 呼出(用件別)～会計クリア

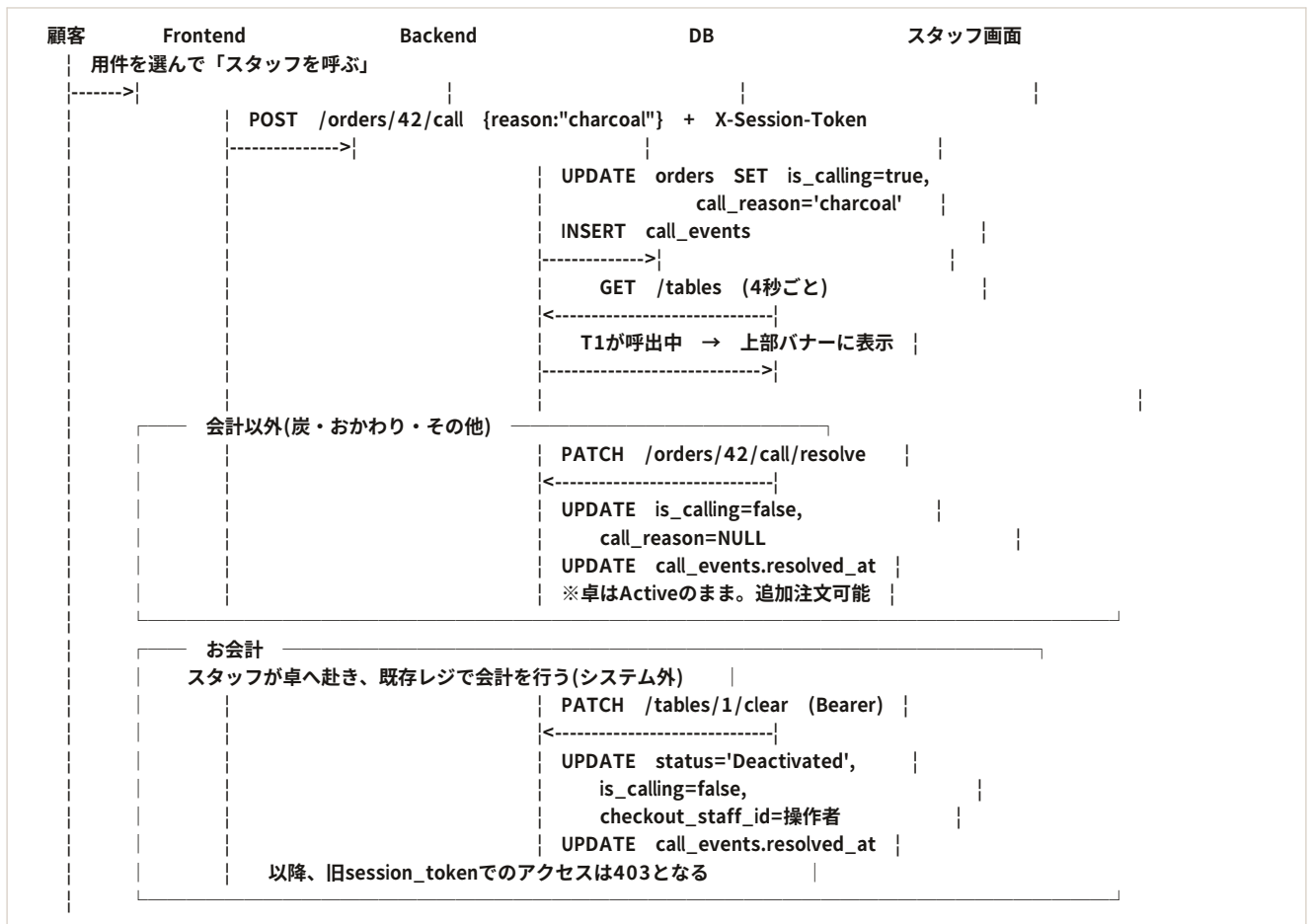


図2 呼出(用件別)～会計クリアシーケンス

3. API詳細定義

3.1 卓セッション取得(Customer)

GET /api/v1/tables/{table_id}/session

```
Response 200:
{
  "order_id": 42,
  "session_token": "a1b2c3d4-e5f6-7890-abcd-ef1234567890",
  "table_number": 1,
  "table_label": "T1",
  "capacity": 5
}
```

- 以降のCustomer系APIのリクエストパスには、必ずこの order_id を使用する。
- 対象卓に status='Active' の伝票が無い場合は 403 (TABLE_NOT_ACTIVE)。
- 同一IPから1分間に10回を超えた場合は 429 (RATE_LIMITED)。

3.2 セッション検証(Customer系API 共通仕様)

GET /orders/{order_id}/history、POST /orders/{order_id}/items、POST /orders/{order_id}/call は、いずれもリクエストヘッダー X-Session-Token を必須とする。共通の処理ロジックは以下のとおり。

- 対象 order_id のレコードを取得し、orders.session_token とヘッダー値が一致するか検証する。
- 不一致、またはヘッダー未指定の場合は 403 (SESSION_EXPIRED) を返し、処理を行わない。
- 対象 order_id の status が 'Deactivated'(会計済)の場合も 403 (SESSION_EXPIRED) を返す。
- この検証はFastAPIの依存性注入(Depends)で共通化し、各エンドポイントでの実装漏れを防ぐ。

3.3 注文追加(Customer)

POST /api/v1/orders/{order_id}/items

```
Request:
{
  "items": [
    { "product_id": 1, "quantity": 1 },
    { "product_id": 4, "quantity": 2 }
  ]
}

Response 201:
{ "created_count": 2 }
```

- product_id から現在の price(税込)を取得し、明細にスナップショットとして保存する。
- is_sold_out = true の商品が含まれる場合は 400 (VALIDATION_ERROR) を返す。
- 複数明細の登録は1トランザクションで行い、途中失敗時は全てロールバックする。

3.4 スタッフ呼出(Customer)

POST /api/v1/orders/{order_id}/call

```
Request:
{ "reason": "charcoal" }           // charcoal | drink | checkout | other

Response 200:
{ "is_calling": true, "reason": "charcoal", "called_at": "2026-08-18T19:42:00+09:00" }
```

- 対象の orders レコードの is_calling を true、call_reason を指定値に UPDATE する。
- 同時に call_events へ1行INSERTする(reason, called_at)。
- reason が列挙値以外の場合は 400 (VALIDATION_ERROR)。
- 既に呼び出し中の場合は reason を上書きし、call_events に新たな行は追加しない(連打対策・冪等)。この場合も 200 を返す。
- ダッシュボードはポーリングによりこのフラグを検知し、上部の呼び出し一覧と卓の赤ドットを表示し続ける。

3.5 呼び出し解除(Staff)

PATCH /api/v1/orders/{order_id}/call/resolve

```
Response 200:
{ "is_calling": false }
```

- orders の is_calling を false、call_reason を NULL に更新する。
- 当該伝票の call_events のうち resolved_at が NULL の最新行に現在時刻を設定する。
- orders.status は変更しない。卓はActiveのままであり、客は引き続き追加注文できる。
- 呼び出し中でない伝票に対して実行された場合も 200 を返す(冪等)。

会計とそれ以外を分ける理由

炭の交換やおかわりは「対応したら呼び出しだけ消えればよい」用件である。ここで会計クリアを使ってしまうと客のセッションが切れ、追加注文ができなくなる。そのため呼び出しの解除と会計クリアは別のAPIとして明確に分離する。画面側でも、会計の用件には解除ボタンを出さない([03]4章)。

3.6 開栓(Staff)

PATCH /api/v1/tables/{table_id}/activate

```
Response 200:
{ "order_id": 43, "session_token": "...", "table_id": 1 }
```

- 新規 order レコードを作成し、session_token (UUID v4) を発行、is_calling を false、call_reason を NULL にする。
- 対象卓に既に status='Active' の伝票が存在する場合、部分UNIQUEインデックス制約によりINSERTが失敗するため、これを捕捉して 409 (TABLE_ALREADY_ACTIVE) を返す。

3.7 会計クリア(Staff)

PATCH /api/v1/tables/{table_id}/clear

- 対象伝票の status を 'Deactivated'、is_calling を false、call_reason を NULL、closed_at を now() に更新し、checkout_staff_id にJWTから取得した操作者のIDを設定する。
- 当該伝票の未解決の call_events に resolved_at を設定する。
- クリア後、旧 session_token でのアクセスは403となる(3.2節)。
- 対象卓に Active な伝票が無い場合は 404 (NOT_FOUND)。

3.8 明細ステータス更新(Staff)

PATCH /api/v1/order_items/{item_id}/status

```
Request:
{ "status": "served" }           // served | pending | cancelled
```

処理ロジック(遷移ルールは[02]3章に準拠):

遷移	可否	応答
pending → served	許可	200
served → pending	許可	200
pending → cancelled	許可	200
served → cancelled	不可	409 (INVALID_STATUS_TRANSITION)
cancelled → 任意	不可	409 (INVALID_STATUS_TRANSITION)

3.9 カスタムアイテム追加(Staff)

POST /api/v1/orders/{order_id}/custom_items

```
Request:
{
  "custom_name": "スペシャルフレーバー",
  "price": 1500,           // 税込
  "quantity": 1           // 省略可。省略時は 1
}
```

- order_items へのINSERT時、product_id = null、is_custom = true として書き込む。マジックナンバー(9999等)は使用しない。
- price は税込で受け取り、そのまま保存する。税率の換算は行わない。
- メニューに存在する商品の価格変更目的では使用しない。その場合は 3.11 の価格更新APIを用いる。

3.10 品切れ設定(Staff)

PATCH /api/v1/staff/products/{id}/sold_out

```
Request:
{ "is_sold_out": true }
```

対象 products.is_sold_out を指定値に更新する。以後、顧客向け GET /products から除外/復帰する。

3.11 メニュー価格更新(Staff)

PATCH /api/v1/staff/products/{id}/price

```
Request:
{ "price": 900 }           // 税込・0以上の整数

Response 200:
{ "id": 89, "name": "GUESTジン", "price": 900 }
```

- 対象 products.price を指定値に更新する。以後、顧客向け GET /products は新しい価格を返す。
- 負値・非整数は 400 (VALIDATION_ERROR)。
- 既存の order_items.price は決して更新しない。注文時点の価格はスナップショットとして保持されるため、価格改定は過去の伝票金額に影響しない([08]7章 不変条件6・10)。実装時にこの点をコメントとして明記すること。

想定される運用

GUESTジンのように仕入れによって価格が変わる商品を、当日の価格に固定するために使う。営業中に更新しても、既に注文された分の金額は動かない。スタッフ向けの手順は [07_利用ガイド] 5章を参照。

3.12 日次強制クリア(Batch)

POST /api/v1/jobs/daily_clear

```
Response 200:
{ "cleared_count": 3 }
```

- リクエストヘッダー X-Cron-Secret を環境変数 CRON_SECRET と照合し、不一致の場合は 401 を返す。
- status='Active' の全伝票を 'Deactivated' に更新し、is_calling を false、call_reason を NULL にする。
- 未解決の call_events に resolved_at を設定する。
- checkout_staff_id は NULL のまま保持する(正規の会計クリアと区別するため)。

3.13 空席状況の取得(Public・認証なし)

GET /api/v1/public/availability

```
Response 200:
{
  "tables": [
    { "table_id": 1, "label": "T1", "capacity": 5, "occupied": true },
    { "table_id": 2, "label": "T2", "capacity": 4, "occupied": false },
    { "table_id": 5, "label": "C1", "capacity": 1, "occupied": false }
  ]
}
```

- 認証ヘッダーは不要。同一IPからの呼び出し頻度のみをレート制限で監視する([01]5章: 1分間に30回まで)。
- tables.is_active = true の卓のみを、table_number の昇順で返す。
- occupied は、対象卓に status='Active' の orders が存在するかどうかの真偽値のみ([02]2.7節のクエリを参照)。
- レスポンスに含めてはならないもの: session_token / order_id / is_calling / call_reason / 金額 / 商品情報。実装時は、これらを持つORMモデルをそのまま返さず、専用のレスポンススキーマ(Pydanticモデル)を介して明示的に絞ったフィールドだけをシリアライズすること。
- Backend側で追加のキャッシュは行わない(V1)。将来アクセスが増えた場合は、数秒程度のインメモリキャッシュを検討する([01]2.5節)。

4. 実装上の共通方針

- リクエスト/レスポンスの型は全てPydanticモデルで定義し、FastAPIの自動生成ドキュメント(/docs)をフロントエンド開発時の参照とする。
- 認証(Bearer / X-Session-Token / X-Cron-Secret)は全てDependsで共通化し、エンドポイント側に検証ロジックを書かない。
- 複数レコードを更新する処理は必ずトランザクション内で実行する。呼び出し関連(orders + call_events)も同様。
- 例外は共通のExceptionHandlerで捕捉し、[01]8章のエラーフォーマットに整形して返す。
- DBアクセスはSQLAlchemy等のORM経由とし、クエリの組み立てを型安全に保つ(方針の詳細は[05]参照)。
- 呼出理由の列挙値は charcoal / drink / checkout / other の4値。Python側はEnumで定義し、DBのCHECK制約と二重に守る。値を追加する場合は [02]5.2節のDDLも同時に変更すること。

- 認証不要のエンドポイント(3.13)は、他と同じレスポンスモデルの仕組みを使いつつも、フィールドをホワイトリスト方式で明示的に絞ったスキーマを別途定義する。内部用のモデルを流用・継承して「除外し忘れる」実装は避ける。

5. 変更履歴

版	主な変更内容
V3(ドラフト)	初版。
V4(レビュー反映)	パスパラメータを table_id に統一。品切れ切替API・明細取消遷移・X-Session-Token検証の共通仕様・開栓の409を追加。
V1.0 (FIX)	シーケンス図2点を追加。未提供リスト取得APIを追加。各APIのリクエスト/レスポンス例と処理ロジックを全面的に具体化。実装上の共通方針を追加。
V1.1	価格を税込で受け渡す方式に変更(tax_rateの授受を廃止)。
V1.2	呼出APIに reason を追加し call_events への記録を規定。呼び出し解除API(3.5)とメニュー価格更新API(3.11)を新設。会計クリア・daily_clearでのcall_reason/call_eventsの扱いを明記。図2を用件別の分岐に更新。
V1.3	空席状況の公開API(3.13)を新設。エンドポイント一覧に「1.1 Public」区分を追加(Customer/Staff/Batchを1つずつ繰り下げ)。レスポンスを絞ったスキーマを使う方針を4章に追記。既存エンドポイントの挙動変更なし。